

Digital Signal Processing Concepts

As Applied in the Image Enhancer / Upscaler Project

Project modules: loader.py | grid.py | interpolation.py | saver.py | enhancer.py

Pipeline: Load -> Decode -> Upsample -> Interpolate -> Sharpen -> Save

Table of Contents

- 1 Sampling & Quantization
- 2 Normalization (ADC Analogy)
- 3 Channel Decomposition
- 4 Sparse Grid & Upsampling
- 5 Interpolation Kernels
 - 5.1 Bicubic (Keys') Kernel
 - 5.2 Lanczos (Windowed Sinc) Kernel
 - 5.3 Separable 2-D Kernels
- 6 Boundary Handling
- 7 Unsharp Mask (Post-Processing)
- 8 Denormalization (DAC Analogy)
- 9 AI Super-Resolution (Real-ESRGAN)
- 10 Full Pipeline Summary

1. Sampling & Quantization

An image is a 2-D discrete signal. When a camera captures a scene, two fundamental DSP operations happen simultaneously:

Spatial Sampling

The continuous light field is sampled at a finite grid of $H \times W$ locations (pixels). Each pixel represents one sample of the scene's intensity at that spatial coordinate. In the project, this discrete grid arrives as a NumPy array of shape $(H, W, 3)$ when loaded via Pillow.

Amplitude Quantization

Each sample's continuous brightness is quantized to one of 256 discrete levels (0-255) and stored as an unsigned 8-bit integer (uint8). This is uniform scalar quantization with 8-bit depth, giving $2^8 = 256$ possible values per channel.

$$Q(x) = \text{round}(x * 255) \rightarrow \text{uint8} \quad [0, 255]$$

Quantization: continuous intensity mapped to 8-bit integer

File: loader.py -> load_image()

```
image = image.convert("RGB")
image_array = np.array(image)    # shape (H, W, 3), dtype uint8
```

2. Normalization (ADC Analogy)

Before any mathematical processing, the integer pixel values are converted to floating-point in the range $[0.0, 1.0]$. This is analogous to an Analog-to-Digital Converter (ADC) producing a normalized continuous representation from discrete samples.

Why is this necessary? Interpolation kernels compute weighted sums of neighbours. Operating in integer space would cause truncation errors (e.g., $127 / 255 = 0$ in integer division). Float64 preserves full precision throughout the pipeline.

$$x_{\text{norm}} = x_{\text{uint8}} / 255.0 \quad \text{float64 in } [0.0, 1.0]$$

Normalization: discrete integers to continuous float space

File: loader.py -> normalize()

```
normalized = channel_array.astype(np.float64) / 255.0
normalized = np.clip(normalized, 0.0, 1.0) # defensive clip
```

3. Channel Decomposition

An RGB image is a vector-valued 2-D signal with three components (Red, Green, Blue). The project decomposes this into three independent scalar signals, each processed through the same pipeline separately.

This is valid because the interpolation kernels are linear and channel-independent: each colour channel can be upsampled in isolation and recombined at the end without cross-channel interference.

$$I(r, c) = [R(r, c), G(r, c), B(r, c)] \rightarrow 3 \text{ independent 2-D signals}$$

File: loader.py -> split_channels()

```
r = image_array[:, :, 0].copy()
g = image_array[:, :, 1].copy()
```

```
b = image_array[:, :, 2].copy()
```

4. Sparse Grid & Upsampling

The core of the upscaling process is creating a larger output grid and placing the known pixel samples at regularly spaced positions. This is a direct implementation of the DSP concept of upsampling (inserting zeros / NaN between samples).

Upsampling by Factor L

Given scale_factor L, the output grid has dimensions (H*L, W*L). Each original pixel at position (r, c) is placed at (r*L, c*L) in the new grid. All other positions are filled with NaN (not zero, because 0.0 is a valid pixel value representing black).

$$\text{sparse}[r*L, c*L] = \text{original}[r, c] \quad \text{all other positions} = \text{NaN}$$

Upsampling rule: place known samples at multiples of L

The fill ratio of the sparse grid is exactly $1/L^2$. For a 4x upscale, only 1/16 of the output pixels are known; the remaining 15/16 must be computed by interpolation. This sparse-to-dense reconstruction is the central DSP challenge of the project.

File: grid.py -> map_pixels()

```
empty_grid = np.full((H*L, W*L), np.nan, dtype=np.float64)
for r in range(H):
    for c in range(W):
        empty_grid[r*L, c*L] = channel[r, c]
```

5. Interpolation Kernels

Interpolation reconstructs the continuous signal from discrete samples, then re-samples it at the new (higher) resolution. The quality of this reconstruction depends entirely on the kernel (also called the interpolation filter) used to weight the contributions of known neighbours.

$$f(x) = \text{SUM}[f(n) * h(x - n)] \quad \text{for all known neighbours } n$$

General interpolation formula: weighted sum of neighbours

5.1 Bicubic (Keys') Kernel

The bicubic kernel is a piecewise cubic polynomial that uses a 4x4 neighbourhood (16 known pixels) around each unknown position. The parameter $a = -0.5$ (Keys' value) gives the best approximation to the ideal sinc reconstructor among all piecewise cubics.

$$h(t) = (a+2)|t|^3 - (a+3)|t|^2 + 1 \quad \text{for } |t| \leq 1 \quad h(t) = a|t|^3 - 5a|t|^2 + 8a|t| - 4a \quad \text{for } 1 < |t| \leq 2$$

Keys' bicubic kernel ($a = -0.5$)

Properties: Continuous first derivative (smooth curve). Compact support (radius = 2). Weights sum to 1 for interior pixels. Produces smooth results with mild blurring.

File: interpolation.py -> bicubic_kernel(t, a=-0.5)

```
abs_t = abs(t)
if abs_t <= 1.0:
    return (a + 2.0)*abs_t**3 - (a + 3.0)*abs_t**2 + 1.0
elif abs_t <= 2.0:
    return a*abs_t**3 - 5.0*a*abs_t**2 + 8.0*a*abs_t - 4.0*a
```

5.2 Lanczos (Windowed Sinc) Kernel

The ideal reconstruction filter is the sinc function, which has infinite support (it never decays to zero). The Lanczos kernel approximates the ideal sinc by windowing it with a scaled copy of itself, truncated to a finite support of radius 'a' (typically 3).

$$L(t) = \text{sinc}(t) * \text{sinc}(t/a) \quad \text{for } |t| < a \quad L(t) = 0$$

Lanczos-a kernel (a=3: 6x6 neighbourhood)

The normalized sinc is defined as $\text{sinc}(x) = \sin(\pi x) / (\pi x)$. NumPy's `np.sinc()` already uses this normalized form.

Properties: Closest practical approximation to ideal reconstruction. Produces sharper output than bicubic. Negative side-lobes cause mild 'ringing' artifacts (values slightly outside [0, 1]) near high-contrast edges. These are clipped during denormalization.

File: interpolation.py -> lanczos_kernel(t, a=3)

```
if t == 0.0: return 1.0
if abs(t) >= a: return 0.0
return float(np.sinc(t) * np.sinc(t / a))
```

5.3 Separable 2-D Kernels

Both bicubic and Lanczos kernels are separable: the 2-D weight for a neighbour at displacement (dr, dc) is the product of two 1-D kernel evaluations. This property is fundamental to efficient implementation.

$$w_{2D}(dr, dc) = h(dr) * h(dc)$$

Separability: 2-D weight = product of two 1-D weights

An N x N neighbourhood with a non-separable kernel requires N² multiplications, while a separable kernel needs only 2N. The project exploits this in `interpolate_pixel()` where `w_row` and `w_col` are computed independently.

File: interpolation.py -> interpolate_pixel()

```
for value, row_dist, col_dist in neighbors:
    w_row = kernel_fn(row_dist)    # 1-D evaluation
    w_col = kernel_fn(col_dist)    # 1-D evaluation
    weight = w_row * w_col         # separability
    weighted_sum += weight * value
```

6. Boundary Handling (Edge Extension)

When interpolating near image edges, the kernel window extends beyond the image boundary. The project uses 'clamp-to-edge' (replicate) padding: out-of-bounds indices are clamped to the nearest valid pixel.

$$n_clamped = \max(0, \min(N-1, n))$$

Clamp-to-edge boundary extension

This is equivalent to extending the border pixels outward infinitely. It avoids introducing artificial dark edges (zero-padding) or discontinuities (wrap-around). SciPy's `map_coordinates` uses `mode='nearest'` which implements identical clamping behaviour.

7. Unsharp Mask (Post-Processing)

Interpolation inherently acts as a low-pass filter: it smooths the image by computing weighted averages. While this eliminates pixelation, it also softens edges. The UnsharpMask is a classic DSP technique to restore perceived sharpness.

Algorithm

- Blur the image with a Gaussian of radius r pixels
- Subtract the blurred version from the original to extract high-frequency edges
- Add a percentage of the extracted edges back to the original
- Only sharpen where the difference exceeds a threshold (suppresses noise)

$$\text{sharpened} = \text{original} + \text{percent}/100 * (\text{original} - \text{blur}(\text{original}))$$

UnsharpMask formula (applied only where $|\text{diff}| > \text{threshold}$)

In DSP terms, this is high-frequency emphasis (high-boost filtering). The Gaussian blur acts as a low-pass filter; subtracting it yields a band-pass (edge) component; adding it back amplifies high frequencies relative to low frequencies.

File: saver.py -> sharpen_image()

```
pil_image = Image.fromarray(image_array, mode="RGB")
sharpened = pil_image.filter(
    ImageFilter.UnsharpMask(
        radius=1.5, percent=120, threshold=3
    )
)
```

8. Denormalization (DAC Analogy)

After interpolation, the float64 values must be converted back to uint8 integers for storage and display. This is analogous to a Digital-to-Analog Converter (DAC).

The process has a careful ordering to avoid artifacts:

- Multiply by 255.0 to scale back to integer range
- Clip to [0, 255] to handle Lanczos ringing overshoot
- Round (not truncate) to avoid systematic darkening bias
- Cast to uint8

$$\text{x_uint8} = \text{uint8}(\text{round}(\text{clip}(\text{x_float} * 255, 0, 255)))$$

Denormalization: float64 back to 8-bit integer

File: saver.py -> denormalize()

```
scaled = channel_array * 255.0
clipped = np.clip(scaled, 0.0, 255.0)
rounded = np.round(clipped)
denormalized = rounded.astype(np.uint8)
```

9. AI Super-Resolution (Real-ESRGAN)

Beyond classical DSP interpolation, the project includes an optional AI-driven super-resolution path using Real-ESRGAN, a deep neural network trained on millions of image pairs.

How It Differs from Classical DSP

- Classical kernels (bicubic, Lanczos) are fixed mathematical functions that rely only on local pixel neighbourhoods.
- Real-ESRGAN uses a learned RRDB (Residual-in-Residual Dense Block) network that captures semantic understanding: it can synthesize plausible texture details that no fixed kernel could produce.
- The GFPGAN face enhancement module adds even more domain-specific knowledge, restoring facial features with dedicated priors.

DSP Connection

Neural super-resolution can be understood as a learned, non-linear, adaptive interpolation filter. Where classical filters apply the same weights everywhere, the network adapts its 'filter' to the local content of each image patch.

10. Full Pipeline Summary

Stage	Module	DSP Operation
LOAD	loader.py	Decode image file -> (H,W,3) uint8 array
VALIDATE	loader.py	Check shape, dtype, range, minimum size
SPLIT	loader.py	Decompose RGB -> 3 independent channels
NORMALIZE	loader.py	uint8 [0,255] -> float64 [0.0,1.0] (ADC)
UPSAMPLE	grid.py	Create (H*L, W*L) sparse grid with NaN gaps
MAP PIXELS	grid.py	Place known samples at (r*L, c*L) positions
INTERPOLATE	interpolation.py	Fill NaN positions via bicubic or Lanczos
DENORMALIZE	saver.py	float64 [0.0,1.0] -> uint8 [0,255] (DAC)
MERGE	saver.py	Recombine R, G, B channels into RGB array
SHARPEN	saver.py	Optional UnsharpMask high-frequency boost
SAVE	saver.py	Write final PNG to disk
AI ENHANCE	enhancer.py	Optional Real-ESRGAN neural upscaling

End of document.